

---

# Documento de Escopo Técnico: SaaS de Agendamento APS (Atenção Primária à Saúde)

## 1. VISÃO GERAL DO PRODUTO

Plataforma Multi-tenant voltada para prefeituras de pequeno e médio porte, com o objetivo de digitalizar a regulação e o agendamento de consultas na Atenção Básica (UBS). O sistema elimina filas físicas através de um App para o cidadão e oferece um painel de gestão para a Secretaria de Saúde e recepção das UBSs.

O escopo do fluxo encerra na recepção da UBS. O ciclo de vida do agendamento termina quando o status do paciente muda para 'Chegou' (A), 'Faltou' (F) ou 'Cancelado' (C).

---

## 2. STACK TECNOLÓGICO E ARQUITETURA

### Programação modular

- **Backend:** Python + FastAPI (Assíncrono, alta performance) .
- **Frontend Gestão (Web):** React.js.
- **Frontend Cidadão (App):** React Native.
- **Banco de Dados:** PostgreSQL (obrigatório suporte a campos `JSONB`).
- **Gestão de Configurações:** Uso de variáveis de ambiente ( `.env` ) para isolamento de credenciais sensíveis e chaves de criptografia.
- **Versionamento de Dados:** Implementação de migrações via Alembic, permitindo a evolução do esquema do PostgreSQL sem perda de integridade dos dados históricos.
- **Comunicação Cross-Origin:** Middleware de CORS configurado para permitir a integração segura entre o Backend e as múltiplas interfaces (Web e Mobile).
- **Background Tasks:** Celery + Redis (ou APScheduler/ARQ) para geração automática de agendas e notificações de lembrete (SMS/Push).
- **High-Performance Buffer (Redis):** Utilização do Redis não apenas para cache e mensageria, mas como um *buffer* em memória de curtíssimo prazo para absorver picos de requisições de gravação (ex: 1000 req/s de logs de auditoria). Os dados são enfileirados no Redis e descarregados no PostgreSQL via *Bulk Insert* assíncrono, protegendo o *Connection Pool* do banco relacional contra exaustão.

---

## 3. CONTROLE DE ACESSO E PERMISSÕES (RBAC + ACL)

O sistema opera com hierarquia baseada no `tenant_id` (ID do Município) e isolamento por `id_ubs`. A injeção de dependência no FastAPI deve garantir essa trava em todas as rotas.

- **Nível 1 (Administrador Master):** Acesso global (Devs/Suporte). Cadastra novos municípios (Tenants).
- **Nível 2 (Gestor Prefeitura):** Preso ao seu `id_municipio`. Gerencia todas as UBSs da cidade, parametriza regras globais (absenteísmo) e cadastra os Gestores Locais (Nível 3).
- **Nível 3 (Usuário UBS / Assistencial):** Preso à sua(s) `id_ubs`. Possui permissões granulares gerenciadas via matriz JSON.
  - **REGRA CRÍTICA (Delegação de Poder):** Se o Nível 2 conceder a permissão `is_gestor_local = true` a um Nível 3, este usuário ganha poderes equivalentes ao Nível 2 **EXCLUSIVAMENTE para a sua UBS**.
  - Ele poderá cadastrar novos recepcionistas/médicos (Nível 3) e definir as permissões deles (ex: liberar criação de escalas), mas o backend do FastAPI deve **travar** o `INSERT` ou `UPDATE` garantindo que o `id_ubs` do novo usuário seja estritamente igual ao `id_ubs` do Gestor Local.

---

## 4. MODELAGEM DE DADOS (ENTIDADES PRINCIPAIS)

O desenvolvedor deve aplicar o conceito de isolamento de Tenant (`id_municipio`) em todas as tabelas transacionais.

### 4.1. Core Multi-tenant e Acessos

- **Municipio (Tenant):** `id_municipio` (PK), `ibge`, `nome`, `config_absenteismo` (JSONB - ex: `{"faltas_limite": 3, "dias_bloqueio": 30}`), `tema_visual` (JSONB).
- **UBS:** `id_ubs` (PK), `id_municipio` (FK), `nome`, `cnes`, `cep`, `endereco`.
- **Profissionais (Usuários):** `id_profissional` (PK), `id_municipio` (FK), `nome`, `cpf` (Unique por Tenant), `conselho`, `senha_hash`, `sn_ativo`. Serve para médicos, enfermeiros e equipe administrativa.
- **Ubs\_Profissionais (Tabela Auxiliar + ACL):** `id_ubs` (FK), `id_profissional` (FK), `permissoes` (JSONB - ex: `{"is_gestor_local": true, "can_create_escala": true}`). *Aqui reside a trava do Nível 3 e a possibilidade dele atuar como admin daquela UBS.*
- **Especialidades:** `id_especialidade`, `nome`, `is_livre_demanda` (Boolean). Se `true`, aparece no App do cidadão.
- **Especialidade\_Profissional:** `id_profissional` (FK), `id_especialidade` (FK).

### 4.2. Pacientes, CAD SUS e ViaCEP

- **Pacientes:** `id_paciente` (PK), `id_municipio` (FK), `id_ubs_referencia` (FK), `nr_cpf`, `nr_cns`, `nm_paciente`, `nm_mae`, `dt_nascimento`, `tp_sexo`, `contato` (JSONB), `sn_ativo`, `is_validado_sus` (Boolean), `dt_aceite_lgpd` (Timestamp).
- **Integração ViaCEP:** Automação do cadastro de endereços (UBS e Pacientes) através de consulta assíncrona ao webservice ViaCEP, garantindo a padronização de logradouros e códigos IBGE.

### 4.3. Motor de Agendamento e Fluxo da Recepção

1. **ESCALA\_CENTRAL (O Molde):** `id_escala` (PK), `id_ubs` (FK), `id_profissional` (FK), `id_especialidade` (FK), `tp_dia_semana` (1-7), `hr_inicio`, `hr_fim`, `qt_atendimento`, `tempo_medio_min` (Crítico para fatiar os slots em memória).
2. **AGENDA\_CENTRAL (O Dia Gerado):** `id_agenda` (PK), `id_escala` (FK), `dt_agenda` (Data exata), `sn_ativo` (Boolean).
3. **Calendário de Feriados:** Integração com BrasilAPI para identificação automática de feriados nacionais e móveis, fornecendo inteligência ao gestor durante a montagem das escalas de trabalho.
4. **IT\_AGENDA\_CENTRAL (O Slot/Agendamento):** `id_item` (PK), `id_agenda` (FK), `hr_agenda` (Hora exata), `id_paciente` (FK, Nullable até marcar), `sn_encaixe` (Boolean), `tp_situacao` (Char) contendo o ciclo de vida exato:
  - **'M' (Marcado):** Paciente agendou. Slot ocupado.
  - **'A' (Chegou/Aguardando):** Fim do fluxo de sucesso. Paciente fez check-in na UBS.
  - **'F' (Faltou):** Fim do fluxo de absenteísmo. Alimentará o bloqueio.
  - **'C' (Cancelado):** Slot volta a ficar livre no App para outros pacientes.

---

## 5. REGRAS DE NEGÓCIO E COMPORTAMENTOS DA API

### 5.1. Integração CAD SUS (Módulo BFF)

- O FastAPI atua como proxy: Rota `GET /api/v1/sus/{cpf_ou_cns}`.
- **Resiliência:** Timeout estrito (ex: 8s). Em caso de falha do Ministério da Saúde, a API retorna HTTP 502/503 com mensagem amigável, e o React Native libera o preenchimento manual dos dados do paciente.
- **Onboarding App:** Se achar no SUS, o front bloqueia a edição de campos oficiais (Nome, Nascimento, Mãe), permitindo apenas a inserção de contato (celular) e o aceite obrigatório da LGPD.

### 5.2. Lógica de Disponibilidade (O App do Cidadão)

- Endpoint: `GET /agendas/disponiveis`.
- O backend fatia o intervalo (`hr_inicio` a `hr_fim` da Escala) baseado no `tempo_medio_min`.
- Cruza com a `IT_AGENDA_CENTRAL` e retorna para o App apenas os slots que não existem na tabela ou que estão com `tp_situacao = 'C'` (Cancelados).
- **Filtros fixos na query:** `is_livre_demanda = True`, `sn_ativo = True`.

### 5.3. Transação de Reserva (Evitando Overbooking)

- No `POST /agendamentos`, o FastAPI deve abrir uma **transação de banco com LOCK** (ex: `SELECT FOR UPDATE` ou Constraint de unicidade).
- O paciente entra com `tp_situacao = 'M'`.

- **Escalas Híbridas (is\_disponivel\_app):** Cada molde de escala possui uma trava de visibilidade. Se desativada, a agenda torna-se exclusiva para marcação presencial na recepção da UBS, sendo ocultada e bloqueada para consultas via App do Cidadão.

## 5.4. Fluxo de Recepção (O Fim da Jornada)

O painel da UBS (React Web) terá uma tela de "Agenda do Dia" operada pelos recepcionistas.

- **Ação de Check-in:** Quando o paciente chega ao balcão, o usuário clica em "Confirmar Presença". O backend dispara `PATCH /agendamentos/{id}` alterando `tp_situacao` para **'A' (Chegou)**. Fim do fluxo.
- **Ação de No-Show:** Ao final do expediente (ou via job noturno), os pacientes que permaneceram como **'M'** (Marcado) devem ter seu status alterado para **'F' (Faltou)**.
- **Automação de Faltas:** Implementação de rotina noturna via APScheduler que processa agendamentos com status **'M'** (Marcado), convertendo-os automaticamente para **'F'** (Faltou) ao fim do dia.

## 5.5. Trava de Absenteísmo Automática

- A trava se baseia exclusivamente no status **'F'**.
- Ao tentar marcar uma nova consulta no App, o FastAPI checa o histórico do `id_paciente`.
- Conta quantos agendamentos estão com status **'F'**. Se atingir o limite configurado na tabela `Municipio` (ex: 3 faltas), recusa o agendamento (HTTP 403) e bloqueia o paciente pela quantidade de dias definida na regra (ex: 30 dias).
- Cancelamentos prévios (**'C'**) no App, com antecedência, não penalizam o cidadão.
- **Fluxo de Cancelamento 'C':** O cidadão pode cancelar agendamentos via App sem penalidades, devolvendo instantaneamente o horário à grade de vagas disponíveis da UBS.

---

# 6. REQUISITOS DE SEGURANÇA E LGPD

1. **Middleware do FastAPI:**
  - **Auditoria:** Gravar log de todos os `POST`, `PUT`, `PATCH` e `DELETE` (Quem fez, IP, Endpoint, Payload alterado). Especialmente quando um Nível 3 cria outro Nível 3.
  - **Tenant Enforcement:** Interceptar todas as rotas operacionais e aplicar `WHERE id_municipio = X`.
2. **Criptografia e Proteção:** Senhas em hash forte (Bcrypt/Argon2). O Token JWT deve conter o `tenant_id`, `id_ubs` padrão e o objeto de `permissoes`.
3. **Termo de Consentimento:** O `POST /pacientes` rejeita requisições do App sem o aceite expresso da LGPD (`dt_aceite_lgpd` não pode ser nulo).

4. Auditoria de Mutação (Arquitetura Desacoplada): Middleware ativo que intercepta todas as operações de escrita (POST, PUT, PATCH, DELETE), capturando o ID do utilizador, IP de origem e rota.

- **Performance:** Para suportar alta simultaneidade sem penalizar o tempo de resposta do utente, o middleware **não grava diretamente no PostgreSQL**. O log em formato JSON é despachado em < 1ms para uma fila no Redis.
- **Persistência (Bulk Insert):** Uma *Background Task* (APScheduler) roda a cada 10 segundos, consome a fila do Redis e realiza um único *Bulk Insert* na tabela `log_auditoria` do PostgreSQL, utilizando apenas 1 conexão com o banco.

## 7. MÓDULO DE RECUPERAÇÃO DE SENHA E NOTIFICAÇÕES (MICROSERVIÇO)

De forma a garantir a escalabilidade e a reutilização do código em futuros produtos da empresa, o fluxo de recuperação de credenciais será dividido numa arquitetura de **Microserviços**. A validação local e o reset manual continuam no *Core* (SaaS Regulação), mas o disparo de mensagens fica a cargo de uma API independente.

### 7.1. API Independente de Mensageria e Identidade (Standalone)

Será desenvolvida uma API separada (ex: `Mensageria-API`), agnóstica ao sistema de saúde, focada exclusivamente na entrega omnicanal e validação de Tokens (OTP).

- **Omnicanalidade:** O utilizador (Cidadão ou Profissional) escolhe por onde quer receber o link ou código de recuperação:
  - **WhatsApp:** Integração via API Oficial (ex: Zenvia, Twilio ou Meta API).
  - **SMS:** Disparo de código OTP curto de 6 dígitos.
  - **E-mail:** Envio de link seguro com token JWT de curta duração (ex: 15 minutos).
- **Agnóstica:** A API recebe apenas o destino (número/email), o canal escolhido e o *payload* (mensagem). Não precisa de saber se é um paciente ou um médico.

### 7.2. Fluxo Self-Service: "Esqueci a minha Senha"

Ação realizada pelo próprio utilizador no App do Cidadão ou na Web (Profissionais).

1. **Solicitação:** O utilizador insere o CPF e clica em "Esqueci a Senha".
2. **Escolha de Canal:** O frontend exhibe os canais disponíveis (ocultando parcialmente os dados por segurança, ex: (31) 9\*\*\*\*-\*\*45 ou `luc***@gmail.com`).

3. **Disparo:** O SaaS Regulação gera um token temporário, assina-o e envia o comando para a **API Independente de Mensageria** disparar a mensagem pelo canal escolhido.
4. **Redefinição:** O utilizador clica no link (ou insere o OTP), valida o token e cadastra uma nova senha (encriptada via Argon2).

### 7.3. Fluxo de Reset Manual (Ação do Gestor / Nível 3)

Para casos em que o cidadão não tem acesso ao telemóvel ou email, o Gestor da UBS (ou Gestor Municipal) pode intervir diretamente no Painel Web.

1. **Comando do Gestor:** O Gestor localiza o Paciente ou Profissional no sistema e clica no botão "Resetar Senha".
2. **Senha Padrão (Fallback):** O sistema altera a `senha_hash` do utilizador para uma senha padrão do sistema (ex: `Cpf@123` ou `Mudar@123`).
3. **Flag de Obrigatoriedade:** O sistema altera uma nova flag na base de dados do utilizador: `is_senha_provisoria = True`.

### 7.4. Trava de Redefinição Obrigatória (First-Login)

Este é um mecanismo crítico de segurança (Zero Trust) para garantir que o Gestor não tem acesso contínuo à conta do paciente/profissional após o reset manual.

1. **Interceção no Login:** Quando o Cidadão (no App) ou o Profissional (na Web) tenta fazer o login com a senha padrão, a rota `POST /login` valida as credenciais.
2. **Verificação da Flag:** O sistema verifica que `is_senha_provisoria == True`.
3. **Bloqueio Parcial (HTTP 403 ou 428):** Em vez de devolver o Token JWT com permissões totais, a API recusa a entrada e devolve um aviso: *"É obrigatório alterar a senha padrão"*. (Alternativamente, devolve um Token restrito que só tem permissão para aceder à rota de alteração de senha).
4. **Ecrã de Nova Senha:** O App/Web redireciona o utilizador para um ecrã onde ele é forçado a digitar a Nova Senha.
5. **Conclusão:** Após a troca, a flag `is_senha_provisoria` volta a `False` e o fluxo normal é restabelecido.

### 7.5. Impacto na Base de Dados (Alembic Migrations)

Para suportar este escopo, as tabelas `Profissionais` e `Pacientes` (caso os pacientes tenham credenciais locais) deverão receber as seguintes novas colunas:

- `is_senha_provisoria` (Boolean): Default `False`. Define se a senha atual foi gerada por um gestor e exige troca imediata.
- `dt_ultimo_reset` (DateTime): Data e hora do último reset para fins de auditoria de segurança.